

Fair Drop — API Contract (v1)

Base: `https://<host>/api/v1` . JSON only. Every response has `Cache-Control: no-store` unless noted.

0. Conventions

Auth kinds

- `none` : public.
- `session` : valid `fd_session` cookie (HMAC-signed `sid.uid.exp`). Worker verifies, then rejects if revoked (D1 lookup cached 30 s per isolate).
- `admin` : `fd_admin` cookie (12 h, from `/admin/login`).
- `sim` : Authorization: Bearer `<sim_token>` , HMAC over `{experiment_id, drop_ids, exp, scope}` . Scope $\in$ `batch` , `control` , `metrics` , `labels` , `finalize` .
- `viewer` : admin cookie **or** `?vt=<viewer token>` (signed, read-only, experiment-scoped, 24 h).

Mutations require header `x-FD-Client: web|sim` (CSRF defence with SameSite cookies) and accept `Idempotency-Key` (UUID). Where a natural key exists, the natural key wins (semantic idempotency).

Error envelope

```
json
{ "error": { "code": "RATE_LIMITED", "message": "...", "retry_after_ms": 4000, "details": {} } }
```

HTTP	code	Meaning
400	VALIDATION_ERROR	Schema failed
401	UNAUTHENTICATED	Missing/invalid/revoked credential
403	FORBIDDEN	Authenticated but not allowed (e.g. sim token on a public drop)
403	CHALLENGE_REQUIRED	Must pass Turnstile first. <code>details.site_key</code>
403	RESTRICTED	Risk action RESTRICT. <code>details.reasons</code> (rule ids only)
404	NOT_FOUND	—
409	INVALID_STATE	Phase/participant state disallows the action. <code>details.state</code>
409	NOT_ALLOCATED	Reserve without a valid offer
410	EXPIRED	Offer/hold expired
422	IDEMPOTENCY_CONFLICT	Same key, different body
429	RATE_LIMITED	<code>retry_after_ms</code>
503	OVERLOADED / HALTED	Load shed / drop halted

Common objects (zod in `packages/shared/schemas.ts`)

```
ts
DropPublic { id, event_id, mode, state, total_inventory, opens_at, registration_closes_at,
             booking_closes_at, seed_commitment, counts:{registered, eligible?, available, held, confirmed}
}
ParticipantView { participant_id, status, registered_at, eligibility_reason?, waitlist_rank?,
                  offer_expires_at?, reservation?: ReservationView, ticket?: {seq, ticket_id},
                  challenge: "NONE"|"REQUIRED"|"PASSED"|"FAILED", next_poll_ms }
ReservationView { reservation_id, status, ticket_seq, expires_at, confirmed_at? }
```

1. Consumer endpoints

#	Method & path	Auth	Request	Response	Idempotency	Errors	Handler
1	GET /events/:eventId	none	—	{event, drops: DropPublic[]}	read	404	Worker → D1 (edge cache 5 s)
2	GET /drops/:dropId	none	—	DropPublic	read	404	Worker → DO getPublic (edge cache 1 s)
3	POST /session	none (cookie optional)	{deviceId, displayName?, email?}	{sessionId, user: {id, displayName}, recovered: bool} + Set-Cookie	Valid cookie → returns the same session (recovered:true). Email given → existing user by email_hash + new session	400, 429 (L0)	Worker → D1
4	GET /session	session	—	{sessionId, user, active drop_ids[]}	read	401	Worker → D1
5	POST /drops/:dropId/register	session	{telemetry?: TelemetrySummary, turnstile token?}	{participant: ParticipantView, deduplicated: bool}	Natural key (drop, account). Repeats return the existing participant with deduplicated:true	403 CHALLENGE_REQUIRED/RESTRICTED, 409 INVALID_STATE (not open), 429	DO register
6	GET /drops/:dropId/me	session	—	ParticipantView + {drop: DropPublic} (covers waiting-room status and allocation status)	read	401, 404 (not a participant → {status:"NONE"} 200 instead), 429	DO me
7	POST /drops/:dropId/telemetry	session	TelemetrySummary (≤ 4 KB)	204	Duplicate window_seq ignored	400, 413, 429 (> 1/10 s)	DO telemetry
8	POST /drops/:dropId/challenge	session	{turnstile token}	{challenge: "PASSED" "FAILED", participant:	Token single-use. Repeat after PASSED → PASSED	400, 409 (no challenge pending), 429	Worker (siteverify) → DO challengeResult

#	Method & path	Auth	Request	Response	Idempotency	Errors	Handler
	/verify			ParticipantView}			
9	POST /drops/:dropId/withdraw	session	—	ParticipantView	Natural	409	DO
10	POST /drops/:dropId/reservations	session	{}	{reservation: ReservationView, created: bool}	Natural key (drop, participant): existing HELD/CONFIRMED reservation returned, created:false	409 NOT_ALLOCATED/INVALID_STATE, 410 EXPIRED (offer), 403 CHALLENGE_REQUIRED/RESTRICTED, 429	DO reserve
11	POST /drops/:dropId/reservations/:resId/confirm	session	{payment token: "mock"}	{reservation, ticket: {seq, ticket_id}}	Natural: already CONFIRMED → same body	404 (not owner → 404, no info leak), 410 EXPIRED, 409	DO confirm
12	POST /drops/:dropId/reservations/:resId/release	session	—	{reservation}	Already RELEASED/EXPIRED → same	404, 409 (CONFIRMED can't be released)	DO release
13	POST /drops/:dropId/offer/decline	session	—	ParticipantView	Natural	409	DO

Expiration has no public endpoint. It's driven by DO alarm / lazy check / virtual clock. Admin can force a sweep (§2).

Polling contract: clients must respect `next_poll_ms` (server sets 2–10 s by phase, ±20 % jitter applied client-side). Polling faster than that hits L1 and adds Network risk points.

2. Admin endpoints (consumer drop control)

Method & path	Auth	Request	Response	Notes / Handler
POST /admin/login	none	{passphrase}	204 + fd_admin cookie	Constant-time compare to ADMIN_PASSPHRASE secret. L0 limit 5/min. Worker
POST /admin/drops	admin	{event id, mode, total inventory, timings, config}	DropPublic (DRAFT)	Worker → D1 + DO init
POST /admin/drops/: id/transition	admin	{to: "SCHEDULED" "CANCELLED" "HALTED" "CLOSED", reason}	DropPublic	Only legal transitions. Clock-driven transitions can be <i>advanced</i> (e.g. close registration now) with {to:"REGISTRATION_CLOSED"}. DO lifecycle
POST /admin/drops/: id/sweep	admin	—	{expired offers, expired holds, promoted}	DO inventory
GET /admin/drops/: id/integrity	admin	—	integrity scan result	DO

3. Simulation endpoints

Creating an experiment requires `admin`. All later simulator calls use the returned `sim` token.

#	Method & path	Auth	Request	Response	Idempotency	Errors	Handler
S1	POST /sim/experiments	admin	{comparison_id?, mode:"FAIR" "NAIVE", sim_mode:"LOGICAL" "REAL_HTTP", scenario, config: ExperimentConfig, population_seed, population_hash, simulator_version}	{experiment_id, drop_id, sim_token, viewer_token, token_expires_at}	Idempotency-Key required → same experiment	400 (config), 403	Worker → D1 + DO init (experiment drop, kind=EXPERIMENT)
S2	POST /sim/experiments/:id/batches	sim:batch	{batch_seq, virtual_now, ops: Op[] (≤ 1000)}	{batch_seq, results: OpResult[] (compact), drop_state, virtual_now}	batch_seq exactly-once: repeated seq → cached response. Gap → 409 BATCH_OUT_OF_ORDER	409, 413, 422	DO simBatch
S3	POST /sim/experiments/:id/control	sim:control	{action: "open" "close_registration" "advance_clock" "close", virtual_now?}	{drop_state, counters}	Natural (state-based)	409	DO lifecycle
S4	POST /sim/experiments/:id/client-metrics	sim:metrics	{t_ms, latency_hist: {buckets_ms:[], counts:[], errors_by_code:{}, requests}	204	(t_ms) upsert	400	DO metrics (REAL_HTTP mode)
S5	POST /sim/experiments/:id/finalize	sim:finalize	{}	{status:"AWAITING_LABELS", integrity, allocation_summary}	Natural	409	DO close → D1
S6	POST /sim/experiments/:id/labels	sim:labels	{chunk_index, chunk_count, labels:[{account_ref, profile, operator_ref}]}	{received, labels_hash?, status}	(chunk_index) upsert	409 if not AWAITING_LABELS. 422 if final hash ≠ population_hash	Worker → D1 compute fairness

#	Method & path	Auth	Request	Response	Idempotency	Errors	Handler
S7	GET /experiment s/:id/live (WebSocket)	viewer	Upgrade	server → {type:"snapshot", counters, rates, latency, integrity} every 500 ms. {type:"event", ts, subject, kind, detail} (≤ 50/s, sampled)	—	401, 429 (> 50 viewers)	Worker → DO live hub
S8	GET /experiments	viewer	? comparison_id=	list	read	—	Worker → D1
S9	GET /experiments/:id/results	none (aggregates only, no PII)	—	Experiment Results (§5)	read	404, 409 (not COMPLETED → partial with status)	Worker → D1
S10	GET /comparisons/:comparisonId	none	—	{fair: ExperimentResults, naive: ExperimentResults, deltas}	read	404	Worker → D1

op (logical mode): compact array form to keep batches small.

```
ts
type Op = {
  t: number;           // virtual ms offset of this attempt
  a: string;           // account_ref (sim-generated, opaque, label-free)
  s: string;           // session_ref
  d: string;           // device_ref
  n: string;           // network_ref (virtual /24)
  op: "session"|"register"|"me"|"telemetry"|"challenge"|"reserve"|"confirm"|"release";
  c?: number;          // amplification count: identical attempts in this tick (default 1, max 10_000)
  tel?: TelemetrySummary; // for telemetry/register
  ch?: "pass"|"fail";   // challenge outcome from the declared oracle (sim drops only)
  k?: string;          // idempotency key
}
type OpResult = [index, httpStatusEquivalent, code, deduplicated?, participantStatus?]
```

Each of the `c` attempts goes through L1–L4 and the risk engine one at a time, and each is counted as a request. **The server never receives profile labels.**

ExperimentConfig (stored verbatim): {total_participants, profile_mix:{HUMAN:0.8, FLOOD_BOT:0.05, ...}, request_amplification, retry_amplification, concurrency, traffic_rate_rps, timings:{registration_ms, offer_ttl_ms, hold_ttl_ms, booking_ms}, capacity_model:{per_tick_ms, capacity_per_tick, selection:"FIFO"}, risk:{enabled, thresholds}, controls:{enabled}, challenge_oracle:{HUMAN:0.97, ...}, confirm_probability:{HUMAN:0.9, ...}}.

4. Audit endpoints

Method & path	Auth	Response	Handler
GET /drops/:dropId/audit	none	AuditRecord (seed revealed only after allocation commit) + snapshot_url	Worker → D1
GET /drops/:dropId/audit/snapshot?chunk=i	none	{chunk index, chunk_count, sha256, participant_ids[]}	Worker → D1 artifacts
POST /audit/verify	none (LO 10/min)	{drop id, participant id?} → {commitment ok, snapshot hash_ok, result hash ok, winners match, integrity ok, participant?: {rank, rank_key, outcome}}	Worker recomputes with packages/shared/allocation.ts (CPU-heavy for 50k, so it runs as a DO RPC on the drop's DO, which has a 30 s CPU budget)

The browser audit page independently re-runs the same algorithm on the downloaded snapshot. The server endpoint is only a convenience.